

# ClubDash Master Engineering Specification

Book a Court (BookCourt.vue): Complete Template Breakdown, JavaScript Function Mappings & Backend API Matrix

## 1. Visual Reference Map: Screenshot to Code Mapping

The live Court Booking interface is mapped to 7 distinct functional areas across the 2 uploaded screenshots:

The image displays two screenshots of the ClubDash 'Book a Court' interface. The first screenshot shows the top section with a sidebar, a header, and a filter bar. The second screenshot shows the bottom section with a grid of court options and a time slot grid. Color-coded boxes are overlaid on the screenshots to indicate specific UI sections and their corresponding code line ranges:

- [1] Page Header & Nearby Courts (Lines 6-35): Indigo box covering the top header and navigation area.
- [2] Club, Date & Period Filters (Lines 40-95): Emerald box covering the filter bar.
- [3] Available Courts Counter & Refresh (Lines 157-177): Sky Blue box covering the available courts counter and refresh button.
- [4] Sport Visual & Badge (Lines 211-223): Amber box covering the sport visual hero and badge.
- [5] Time Slots Grid: Past vs Available (Lines 251-281): Rose box covering the time slots grid.
- [6] Dynamic Booking Button CTA (Lines 291-315): Purple box covering the dynamic booking button.

| Box #        | Book Court UI Section          | Template Line Range | Core Responsibilities & State                                                   |
|--------------|--------------------------------|---------------------|---------------------------------------------------------------------------------|
| [1] Indigo   | Page Header & GPS Link         | Lines 6 – 35        | Renders title & /member/nearby-courts navigation button.                        |
| [2] Emerald  | Club, Date & Period Filter Bar | Lines 40 – 95       | Binds selectedClubId, selectedDate, & selectedPeriod.                           |
| [3] Sky Blue | Available Courts Counter       | Lines 157 – 177     | Displays total active court count & manual data refresh button.                 |
| [4] Amber    | Sport Visual Hero & Badge      | Lines 211 – 223     | Renders high-res sport image & pill badge (Tennis, Badminton, etc.).            |
| [5] Rose     | Time Slots Grid (Past vs Open) | Lines 251 – 281     | Calculates availability matrix; greys out past hours with 'Past' label.         |
| [6] Purple   | Dynamic Booking CTA Button     | Lines 291 – 315     | Anti-double-click guard; confirms slot and forwards to Stripe checkout.         |
| [7] Modal    | Success / Payment Dialog       | Lines 321 – 370     | Displays confirmation modal with 'Proceed to Payment' & 'Book Another' buttons. |

2. Master Data Origin & API Flow Architecture

| UI Variable           | Data Source Type                    | Exact Backend API Endpoint                       | Backend Controller & Service                                     | PostgreSQL Table                               |
|-----------------------|-------------------------------------|--------------------------------------------------|------------------------------------------------------------------|------------------------------------------------|
| clubs                 | Reactive Ref (clubs.value)          | GET /api/v1/clubs                                | app/clubs/controllers.py<br>ClubService.get_all_clubs            | clubs<br>(All active arenas)                   |
| courts                | Reactive Ref (courts.value)         | GET /api/v1/clubs/<id>/courts                    | app/clubs/controllers.py<br>CourtService.get_club_courts         | courts<br>(is_active=True)                     |
| availability (Matrix) | Reactive Ref (availability.value)   | GET /api/v1/availability/matrix?club_id=X&date=Y | app/clubs/controllers.py<br>CourtService.get_availability_matrix | courts cross-referenced with bookings          |
| createBooking         | Pinia Action / Axios (bookingStore) | POST /api/v1/bookings                            | app/bookings/controllers.py<br>BookingService.create_booking     | bookings<br>(Creates row with status 'active') |
| selectedClubId        | Two-way Binding (v-model)           | Client-side ref initialized on load              | N/A (Local UI State)                                             | N/A                                            |
| selectedDate          | Two-way Binding (v-model)           | Defaults to today computed                       | N/A (Local UI State)                                             | N/A                                            |

3. Availability Matrix Architecture: 2D Grid Representation

**Why is it called a 'Matrix'?** Because it computes a 2-Dimensional intersecting grid of **Courts (Rows ↓)** by **Time Slots (Columns ↔)**. The Flask backend calculates operating hours (`06:00` to `22:00`), slices them by `slot\_duration\_minutes`, and checks existing reservations in PostgreSQL:

```
// JSON Response from GET /api/v1/availability/matrix?club_id=1&date=2026-08-29
{
  "date": "2026-08-29",
  "courts": [
    {
      "court_id": 1,
      "court_name": "Tennis Court 1",
      "slots": [
        { "start_time": "09:45:00", "end_time": "10:45:00", "status": "available", "reason": null },
        { "start_time": "10:45:00", "end_time": "11:45:00", "status": "booked", "reason": null },
        { "start_time": "11:45:00", "end_time": "12:45:00", "status": "blocked", "reason": "admin_block" }
      ]
    }
  ]
}
```

4. Master JavaScript Function & API Binding Traceability Reference

This comprehensive table connects every JavaScript method, state mutation, and API call directly to where and how it is used in the template:

| Function / Method    | Where Used in Template                                                              | How & Why It Is Used (Runtime Logic)                                                                                                                      | API & Data Flow                                                                             |
|----------------------|-------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------|
| handleClubChange()   | Line 45:<br><select @change="handleClubChange">                                     | Triggered when player changes the club dropdown. Resets stale courts, clears selectedSlot, and sequentially triggers loadCourts() and loadAvailability(). | Fires:<br>• GET /api/v1/clubs/<id>/courts<br>• GET /api/v1/availability/matrix              |
| loadAvailability()   | Line 53:<br><input @change="loadAvailability"><br>Line 112:<br>@click="refreshData" | Clears selectedSlot.value = null, sets loading.value = true, and requests the full 2D slot availability matrix for the chosen date.                       | Fires:<br>GET /api/v1/availability/matrix?club_id=X&date=Y<br>Populates: availability.value |
| selectPeriod(period) | Line 61:<br><button @click="selectPeriod(period.value)">                            | Updates selectedPeriod.value to 'all', 'morning', 'afternoon', or 'evening'. Clears active slot selection to prevent booking a hidden slot.               | Client-side reactive filter; triggers re-evaluation of getVisibleSlots().                   |
| getCourtImage(court) | Line 205:<br>                                       | Normalizes court.sport_type and looks up local dictionary (e.g. tennis → tennisCourtImage). Falls back to multiPurposeCourtImage.                         | Client-side asset resolver (Vite production bundle hash).                                   |

|                             |                                                                               |                                                                                                                                                                        |                                                                                                |
|-----------------------------|-------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------|
| formatSportName(type)       | Line 206:<br>{ formatSportName(court.sport_type) }                            | Replaces hyphens with spaces and capitalizes words to render clean badge text like <b>"Tennis"</b> or <b>"Multi-purpose"</b> .                                         | Client-side text transformer.                                                                  |
| handleCourtImageError()     | Line 205:<br><img @error="handleCourtImageError">                             | Error fallback handler: If court photo fails to load over CDN, sets event.target.src to default multi-purpose image.                                                   | DOM error event catcher.                                                                       |
| availableSlotCount(court)   | Line 214:<br>{ availableSlotCount(court) }<br>available                       | Counts remaining open slots for court by filtering slot.available && !isSlotInPast(selectedDate, slot.start_time).                                                     | Renders live count badge (e.g. <b>"6 available"</b> or <b>"0 available"</b> ).                 |
| getVisibleSlots(court)      | Line 218:<br>v-for="slot in getVisibleSlots(court)"                           | Extracts court slots from matrix and filters by active period (Morning <12:00, Afternoon 12:00–17:00, Evening ≥17:00).                                                 | Generates the 2-column slot button list in the UI.                                             |
| isSlotInPast(date, start)   | Line 219: :class="{ 'slot-unavailable': ... }"<br>Line 220: :disabled="..."   | Constructs a JS Date object from date and start_time. Compares against currentTime.value. If slot has elapsed, disables button.                                        | Prevents users from selecting or booking time slots in the past.                               |
| slotStatusText(slot)        | Line 225:<br>{ slotStatusText(slot) }                                         | Returns label for disabled slots: <b>"Past"</b> (elapsed today), <b>"Booked"</b> (reserved by another user), <b>"Blocked"</b> (maintenance), or <b>"Unavailable"</b> . | Provides clear visual feedback why a slot is unclickable.                                      |
| selectSlot(court, slot)     | Line 221:<br><button @click="selectSlot(court, slot)">                        | Validates that slot is open and not in the past. Sets selectedSlot.value = { courtId, start_time, end_time }.                                                          | Activates indigo ring around court and enables the 'Book' CTA button.                          |
| isSelectedSlot(court, slot) | Line 219:<br>:class="{ 'slot-selected': ... }"                                | Boolean check: Returns true if this specific slot matches selectedSlot.value, applying active styling.                                                                 | Visual highlight state.                                                                        |
| bookCourt(court)            | Line 231:<br><button @click="bookCourt(court)">                               | Locks booking.value = true to block rapid double-clicks. Slices timestamps to HH:MM, executes bookingStore.createBooking(payload). On success, opens modal.            | Dispatches: POST /api/v1/bookings<br>Payload: { court_id, booking_date, start_time, end_time } |
| proceedToPayment()          | Line 331:<br><button @click="proceedToPayment">                               | Routes player to /member/checkout?payment_type=booking&reference_id=X where Stripe card payment takes place.                                                           | Vue Router navigation to Cashier checkout.                                                     |
| closeSuccess()              | Line 323:<br>@click.self="closeSuccess"<br>Line 332:<br>@click="closeSuccess" | Closes confirmation modal, resets state, and allows player to book additional court slots.                                                                             | Client-side modal dismissal.                                                                   |

■ 5. Complete Template Deep Dive (Lines 1 to 370)

A. [Box 1] Page Heading & GPS Navigation (Lines 6 – 35)

```
6: <div class="flex flex-col gap-4 sm:flex-row sm:items-end sm:justify-between">
7:   <div>
8:     <h1 class="text-2xl font-bold gradient-text">Book a Court</h1>
9:     <p class="text-sm text-gray-500 mt-1">Find an available court and reserve your preferred time slot.</p>
10:   </div>
11:   <router-link to="/member/nearby-courts" class="btn-secondary inline-flex items-center justify-center gap-2">
12:     <svg class="h-4 w-4 text-indigo-600" ...></svg>
13:     Find nearby courts
14:   </router-link>
15: </div>
```

| Line #      | Exact Code Snippet                                             | What It Renders & Does in the Live UI                                                        |
|-------------|----------------------------------------------------------------|----------------------------------------------------------------------------------------------|
| Line 8      | <h1 class="text-2xl font-bold gradient-text">Book a Court</h1> | Renders the bold indigo-gradient page title <b>"Book a Court"</b> .                          |
| Lines 9–10  | <p class="text-sm text-gray-500 mt-1">...</p>                  | Renders the subtitle: <i>"Find an available court and reserve your preferred time slot."</i> |
| Lines 11–14 | <router-link to="/member/nearby-courts">...</router-link>      | Renders the secondary button with location pin icon navigating to GPS court finder.          |

B. [Box 2] Interactive Filter Bar (Lines 40 – 95)

```
40: <div class="glass p-5">
41:   <div class="grid grid-cols-1 md:grid-cols-2 gap-5">
42:     <!-- Select Club Dropdown -->
43:     <div>
44:       <label class="block text-xs text-gray-500 mb-2">Select Club</label>
45:       <select v-model="selectedClubId" class="field" :disabled="booking" @change="handleClubChange">
46:         <option value="">Select a club</option>
47:         <option v-for="club in clubs" :key="club.id" :value="club.id">{{ club.name }}</option>
48:       </select>
49:     </div>
50:     <!-- Booking Date Picker -->
51:     <div>
52:       <label class="block text-xs text-gray-500 mb-2">Booking Date</label>
53:       <input v-model="selectedDate" type="date" :min="today" class="field" :disabled="booking" @change="loadAvailab
ility" />
54:     </div>
55:   </div>
56:   <!-- Time of Day Filter Pills -->
57:   <div class="mt-5">
58:     <label class="block text-xs text-gray-500 mb-3">Time of Day</label>
59:     <div class="flex flex-wrap gap-2">
60:       <button v-for="period in periods" :key="period.value" type="button" class="period-button"
61:         :class="{ 'period-button-active': selectedPeriod === period.value }" @click="selectPeriod(period.valu
e)">
62:         {{ period.label }}
63:       </button>
64:     </div>
65:   </div>
66: </div>
```

| Line #      | Exact Code Snippet                                                                                    | What It Renders & Does in the Live UI                                                                    |
|-------------|-------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------|
| Lines 45–48 | <code>&lt;select v-model="selectedClubId" @change="handleClubChange"&gt;</code>                       | Binds selectedClubId. Selecting 'ClubDash Sports Arena' triggers handleClubChange().                     |
| Line 53     | <code>&lt;input v-model="selectedDate" type="date" :min="today" @change="loadAvailability"&gt;</code> | Binds selectedDate locked to :min="today". Selecting a date triggers loadAvailability().                 |
| Lines 60–63 | <code>v-for="period in periods" :class="{ 'period-button-active': ... }"</code>                       | Renders period pills (All Day, Morning, Afternoon, Evening). Active filter glows with indigo background. |

## C. [Boxes 4, 5, 6] Court Cards, Time Slots Grid &amp; Booking CTA (Lines 200 – 315)

```

200: <div class="grid grid-cols-1 md:grid-cols-2 xl:grid-cols-3 gap-5">
201:   <div v-for="court in activeCourts" :key="court.id" class="glass overflow-hidden
202:     :class="{ 'ring-2 ring-indigo-500 shadow-lg': String(selectedSlot?.courtId) === String(court.id) }">
203:     <!-- Sport Image & Badge -->
204:     <div class="court-image-shell">
205:       
206:       <div class="court-image-badge">{{ formatSportName(court.sport_type) }}</div>
207:     </div>
208:     <div class="p-5">
209:       <div class="flex justify-between items-start">
210:         <h3 class="font-semibold text-gray-200">{{ court.name }}</h3>
211:         <span class="px-2.5 py-1 rounded-full text-[11px] bg-emerald-500/10 text-emerald-400">Available</span>
212:       </div>
213:       <div class="flex justify-between mt-5 mb-3 text-xs text-gray-400">
214:         <span>Available Slots</span><span>{{ availableSlotCount(court) }} available</span>
215:       </div>
216:       <!-- Time Slots 2-Column Grid -->
217:       <div class="grid grid-cols-2 gap-2">
218:         <button v-for="slot in getVisibleSlots(court)" :key="slot.start_time" type="button" class="slot-button"
219:           :class="{ 'slot-selected': isSelectedSlot(court, slot), 'slot-unavailable': !slot.available || isSlotInPast(selectedDate, slot.start_time) }"
220:           :disabled="!slot.available || isSlotInPast(selectedDate, slot.start_time) || booking"
221:           @click="selectSlot(court, slot)">
222:           <div>{{ formatTime(slot.start_time) }}</div>
223:           <div class="text-[10px] opacity-50">to {{ formatTime(slot.end_time) }}</div>
224:           <div v-if="!slot.available || isSlotInPast(selectedDate, slot.start_time)" class="text-[9px] opacity-70">
225:             {{ slotStatusText(slot) }}
226:           </div>
227:         </button>
228:       </div>
229:       <!-- Booking Button CTA -->
230:       <button type="button" class="btn-primary w-full mt-5" :disabled="booking || !selectedSlot || String(selectedSlot?.courtId) !== String(court.id)"
231:         @click="bookCourt(court)">
232:         {{ selectedSlot && String(selectedSlot?.courtId) === String(court.id) ? `Book ${formatTime(selectedSlot.start_time)}` : 'Select a Time Slot' }}
233:       </button>
234:     </div>
235:   </div>
236: </div>

```

| Line #        | Exact Code Snippet                                                         | What It Renders & Does in the Live UI                                                                                      |
|---------------|----------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------|
| Line 202      | :class="{ 'ring-2 ring-indigo-500 shadow-lg': ... }"                       | Applies a vibrant indigo border ring around the court card when a slot on that court is clicked.                           |
| Lines 205–206 |  + badge                                  | Renders the high-res sport cover image and bottom-left badge (e.g. <b>TENNIS</b> , <b>BADMINTON</b> , <b>BASKETBALL</b> ). |
| Line 210      | {{ court.name }}                                                           | Renders court name (e.g. "Tennis Court 1", "badminton 2", "BasketBall court 1").                                           |
| Line 214      | {{ availableSlotCount(court) }} available                                  | Outputs total active slots count (e.g. "6 available" or "0 available").                                                    |
| Lines 218–221 | v-for="slot in getVisibleSlots(court)"<br>@click="selectSlot(court, slot)" | Renders 2-column grid of time slots. Clicking an open green slot highlights it and updates selectedSlot.                   |
| Lines 224–226 | {{ slotStatusText(slot) }}                                                 | Outputs "Past" in muted gray for elapsed hours today, or "Booked" if reserved by another user.                             |
| Lines 230–233 | <button class="btn-primary">                                               | Renders "Book 3:45 PM" (when selected) or disabled "Select a Time Slot".                                                   |

D. [Box 7] Success Confirmation & Payment Modal (Lines 321 – 370)

```
323: <div v-if="successMessage" class="fixed inset-0 z-50 flex items-center justify-center bg-black/70 px-4" @click.self="closeSuccess">
324:   <div class="glass max-w-md w-full p-7 text-center">
325:     <div class="w-14 h-14 mx-auto rounded-full bg-emerald-500/10 flex items-center justify-center">
326:       <svg class="w-7 h-7 text-emerald-400" ...></svg>
327:     </div>
328:     <h3 class="text-xl font-semibold text-gray-900 mt-4">Court Reserved!</h3>
329:     <p class="text-sm text-gray-600 mt-2">{{ successMessage }}</p>
330:     <div class="mt-6 flex flex-col gap-3 sm:flex-row">
331:       <button class="btn-primary flex-1" @click="proceedToPayment">Proceed to Payment</button>
332:       <button class="btn-secondary flex-1" @click="closeSuccess">Book Another Court</button>
333:     </div>
334:   </div>
335: </div>
```

| Line #       | Exact Code Snippet                                  | What It Renders & Does in the Live UI                                                          |
|--------------|-----------------------------------------------------|------------------------------------------------------------------------------------------------|
| Line 323     | v-if="successMessage"<br>@click.self="closeSuccess" | Renders full-screen backdrop overlay when booking succeeds. Clicking outside closes the modal. |
| Line 328–329 | <h3>Court Reserved!</h3> {{ successMessage }}       | Displays green checkmark and booking confirmation details.                                     |
| Line 331     | @click="proceedToPayment"                           | Navigates user to /member/checkout?payment_type=booking&reference_id=X for Stripe checkout.    |
| Line 332     | @click="closeSuccess"                               | Closes modal and resets state so user can book additional courts.                              |